

第 50 讲：LSTM 与 GRU

门控结构到底怎样缓解普通 RNN 的长期依赖问题

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

第 49 讲已经建立了普通 RNN 的核心心智模型：当前 Hidden State 由当前输入与上一时刻 Hidden State 共同决定，同一套参数沿时间反复使用；它也因此必须通过时间展开做 BPTT，并容易出现长期梯度消失/爆炸。本讲学习 LSTM 和 GRU，不是为了背“几个门”，而是回答：**为什么增加一个更受控的状态通道和门控机制，可以让信息与梯度更容易跨越更长时间？**

0. 学完以后，你应该能做到什么？

- 区分 LSTM 的 Hidden State h_t 与 Cell State c_t 。
- 解释 Forget / Input / Output Gate 分别控制什么。
- 手算一个最小 LSTM 的 $i_t, f_t, g_t, o_t, c_t, h_t$ 。
- 解释为什么 Cell State 的加法更新比普通 RNN 的反复非线性变换更利于长期梯度传播。
- 理解“ $\partial c_t / \partial c_{t-1} = f_t$ ”只是直接 Cell-State 路径的局部直觉，并不是完整总导数。
- 看懂 PyTorch `nn.LSTM` 的输入、输出、 h_n, c_n Shape。
- 解释 GRU 的 Reset Gate 与 Update Gate，并与 LSTM 比较。
- 理解 RNN / GRU / LSTM 参数量为什么约为 1 : 3 : 4。
- 理解状态跨 chunk 传递、Truncated BPTT 与流式处理的关系。
- 解释 LSTM / GRU 缓解了长期依赖问题，但没有解决时间轴串行计算这一结构瓶颈。

【科研连接】

你以前在 LLM 课程里已经见过 LSTM / GRU，并知道 Transformer 后来成为语言模型主流。这一次我们不从“模型史”背结论，而是从计算图内部回答：**LSTM 到底改了哪一条信息路径？GRU 又删掉了什么？这些改变为什么能影响 Gradient Flow？**

1 先回顾普通 RNN 的真正困难：历史必须不断穿过同一个非线性瓶颈

普通 RNN：

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b)$$

看起来只是一个很短的公式，但如果把 h_T 展开，它会间接依赖：

$$h_{T-1}, h_{T-2}, \dots, h_1, h_0$$

Backward 时，梯度也要沿这些时间步一路乘回来。

如果每一步的局部导数绝对值经常小于 1，那么：

$$0.8^{50} \approx 1.43 \times 10^{-5}$$

如果经常大于 1，则可能反过来指数放大。

【通俗解释】

普通 RNN 的问题并不是“完全没有记忆”，而是它把历史反复压进同一个 Hidden State，并在每个时间步都经过 Weight 和非线性。长期信息要连续穿过很多道变换，长期 Gradient 也必须穿过同样长的计算路径。

2 LSTM 的核心设计：把“内部记忆通道”与“当前对外输出”分开

LSTM 的全称是 **Long Short-Term Memory**。与普通 RNN 只有一个核心状态 h_t 不同，LSTM 显式维护两类状态：

- c_t ： **Cell State**，可理解为更直接的内部记忆通道；
- h_t ： **Hidden State**，当前时间步对外暴露的表示，同时参与下一步 Gate 计算。

【原理】

LSTM 的关键不是“存储无限长历史”，而是引入：

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

旧 Cell State 不必每一步都完全重写，而是可以经过 Forget Gate 做近似线性、加法式地保留，再加上新的候选记忆。

这条加法路径是理解 LSTM 的第一核心。

3 LSTM 的四个 Gate / Candidate 到底是什么？

为了接近 PyTorch，我们写成：

$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

$$g_t = \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg})$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

然后：

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh(c_t)$$

- f_t Forget Gate: 旧记忆保留多少；
- i_t Input Gate: 新候选信息写入多少；
- g_t Candidate: 准备写入的新内容；
- o_t Output Gate: Cell State 有多少暴露为当前 Hidden State。

【常见误区 / 风险】

Gate 不是硬开关。Sigmoid 输出通常是 0 到 1 之间的连续数，因此“忘掉多少”“写入多少”“输出多少”通常是软比例，而不是简单的 True / False。

4 一个最小 LSTM 手算：每个数字到底发生了什么？

取标量：

$$x_t = 0.7, \quad h_{t-1} = 0.2, \quad c_{t-1} = 0.5$$

实验中人为设置四组 Weight / Bias，得到：

$$i_t \approx 0.627148$$

$$f_t \approx 0.694236$$

$$g_t \approx 0.362707$$

$$o_t \approx 0.576885$$

于是旧记忆保留项：

$$f_t c_{t-1} \approx 0.694236 \times 0.5 \approx 0.347118$$

新记忆写入项：

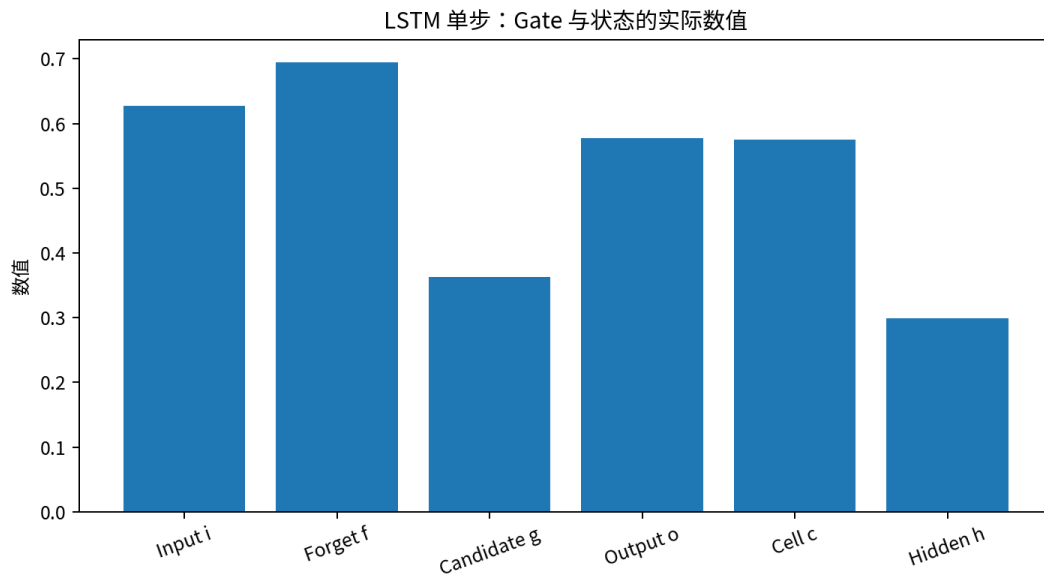
$$i_t g_t \approx 0.627148 \times 0.362707 \approx 0.227471$$

Cell State：

$$c_t \approx 0.347118 + 0.227471 = 0.574589$$

Hidden State：

$$h_t = o_t \tanh(c_t) \approx 0.299243$$



手算与 `PyTorch nn.LSTMCell(1,1)` 的差异只有约 10^{-8} 。

【通俗解释】

真正应该看的不是 0.574589 这个具体数字，而是 Cell State 的结构：

旧记忆保留 + 新信息写入

这和普通 RNN “把上一 Hidden State 与当前输入重新塞进一个 $\tanh$ ” 是不同的状态更新形式。

5 为什么 Forget Gate 是长期记忆的关键旋钮？

先隔离最简单情况：没有新信息写入，只看：

$$c_t = f c_{t-1}$$

连续 T 步：

$$c_T = f^T c_0$$

如果：

$$f = 0.5$$

60 步后几乎完全消失；但：

$$f = 0.99$$

60 步后仍保留：

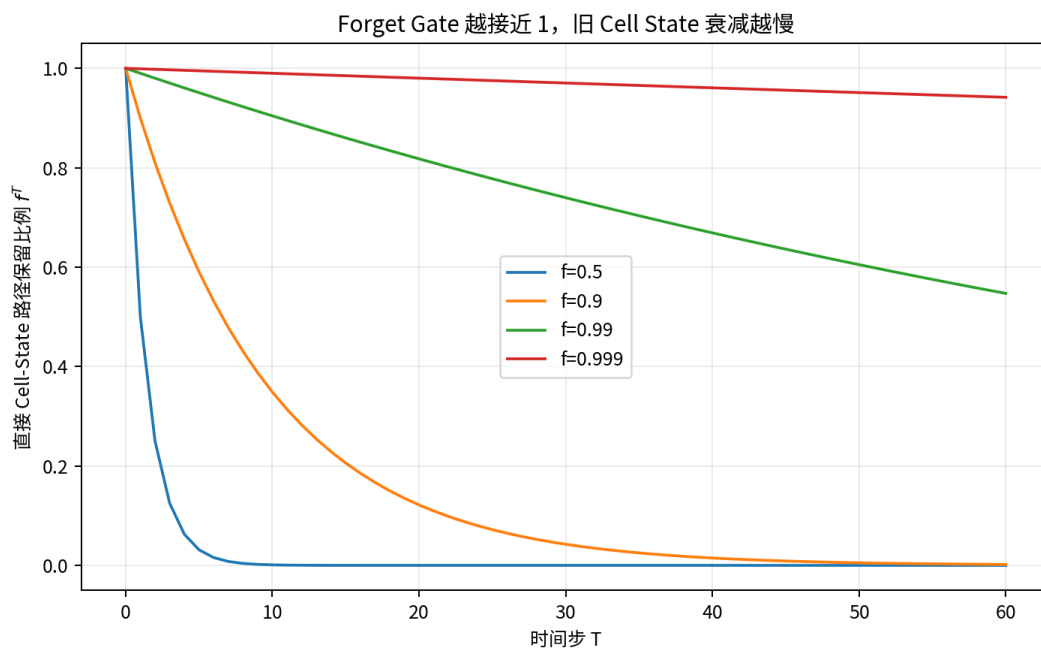
$$0.99^{60} \approx 0.547$$

如果：

$$f = 0.999$$

60 步后仍约：

$$0.942$$

**【常见误区 / 风险】**

这不是说真实 LSTM 会永远把 Forget Gate 固定在 0.99。Gate 是由数据和参数动态计算的。这个实验只是隔离一个变量，让你看到：**Gate 越接近 1, Cell State 的直接传递路径衰减越慢。**

6 LSTM 为什么能缓解 Vanishing Gradient? 关键看直接 Cell-State 路径

由：

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

如果只看 $c_{t-1} \rightarrow c_t$ 这条直接路径，并把当前 Gate 输出视为局部固定量：

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

多步直接路径：

$$\frac{\partial c_T}{\partial c_0} \approx \prod_{t=1}^T f_t$$

如果很多 f_t 接近 1，这条 Gradient Path 可以比普通 RNN 中反复穿过 W_{hh} 与 $\tanh'$ 更容易保存。

实验比较：

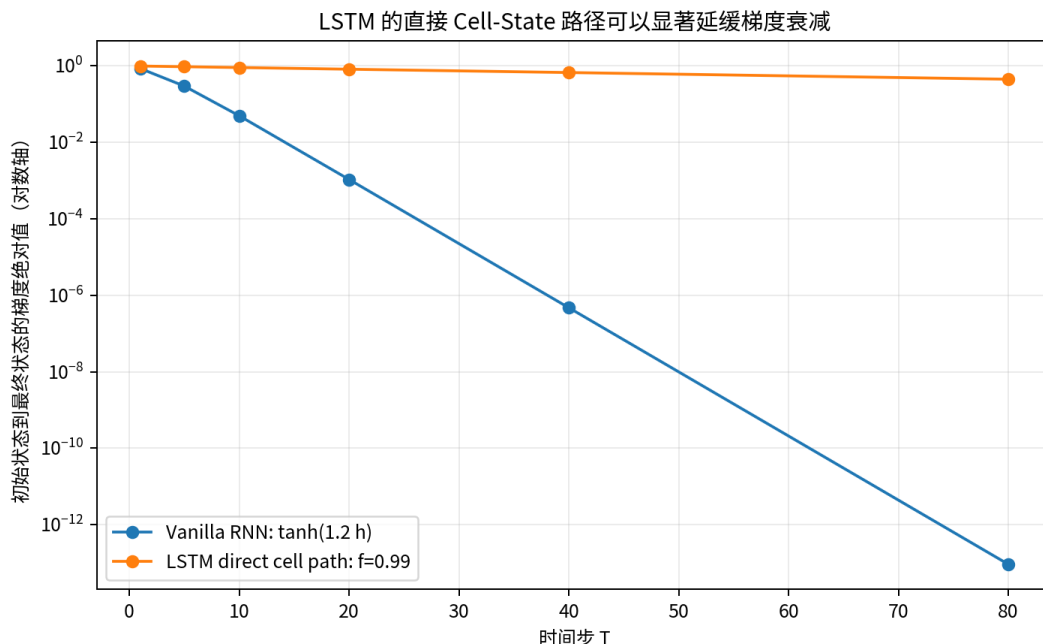
- 普通 RNN: $h_t = \tanh(1.2h_{t-1})$;
- LSTM 直接记忆路径: $c_t = 0.99c_{t-1}$ 。

80 步后：

$$\left| \frac{\partial h_T}{\partial h_0} \right| \approx 9.16 \times 10^{-14}$$

而：

$$\left| \frac{\partial c_T}{\partial c_0} \right| \approx 0.4475$$



【常见误区 / 风险】

严格来说，真实 LSTM 的 Gate 本身也依赖 h_{t-1} ，而 h_{t-1} 又依赖更早状态，因此总导数不只有 f_t 这一条路径。 $\partial c_t / \partial c_{t-1} = f_t$ 是理解**直接 Cell-State 通路**的局部核心，而不是对完整复杂计算图总梯度的全部描述。

7 LSTM 并没有“彻底解决”长期依赖

LSTM 通常比普通 RNN 更容易学习长期关系，但不能推出：

LSTM $\Rightarrow$ 无限长期记忆

原因包括：

- Forget Gate 长期小于 1 时仍会连续衰减；
- Gate 可能进入饱和区域；
- Cell State 维度仍然有限；
- BPTT 路径依然很长；
- 时间轴依然必须递推。

这正是后面 Transformer 仍然会进一步改变序列结构的重要背景。

8 PyTorch LSTM 的参数 Shape：为什么参数量大约是普通 RNN 的 4 倍？

对单层、单向 LSTM：

$$x_t \in \mathbb{R}^D, \quad h_t, c_t \in \mathbb{R}^H$$

PyTorch 把四组 Gate 参数堆在一起，因此：

$$\text{weight_ih_l0.shape} = (4H, D)$$

$$\text{weight_hh_l0.shape} = (4H, H)$$

$$\text{bias_ih_l0.shape} = (4H)$$

$$\text{bias_hh_l0.shape} = (4H)$$

因此在 bias=True 时，参数量：

$$4(HD + H^2 + 2H)$$

同样条件下，普通 RNN 参数量：

$$HD + H^2 + 2H$$

GRU 约为：

$$3(HD + H^2 + 2H)$$

实验取：

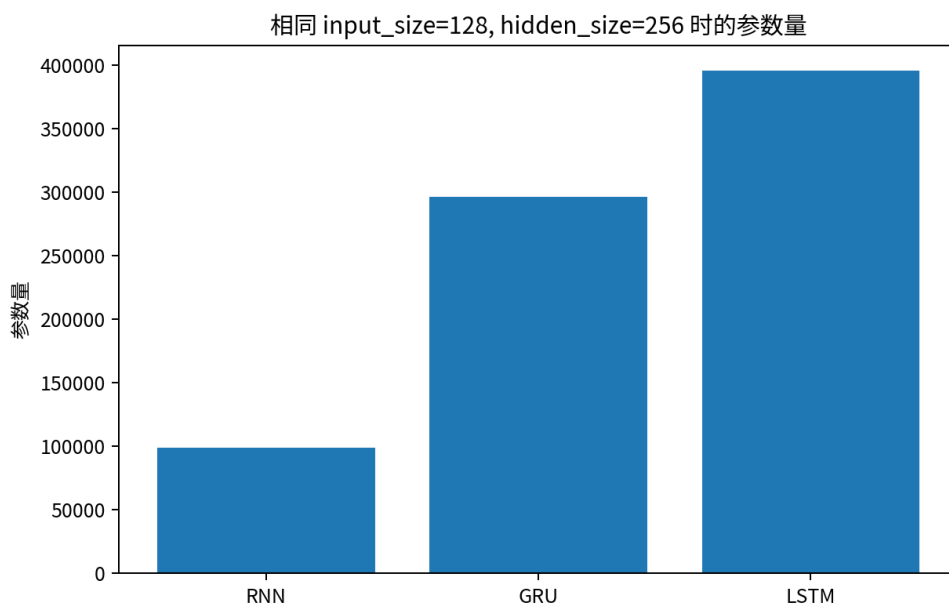
$$D = 128, \quad H = 256$$

得到：

$$RNN = 98,816$$

$$GRU = 296,448$$

$$LSTM = 395,264$$



【通俗解释】

LSTM 更强的门控不是免费的。每个时间步要做更多 Linear、Sigmoid、tanh 和逐元素运算，因此 Parameter、计算量和状态量都会增加。

9 PyTorch nn.LSTM 的输入输出 Shape

设置：

```
nn.LSTM(input_size=D, hidden_size=H, batch_first=True)
```

输入：

$$X : (B, T, D)$$

输出：

$$output : (B, T, H)$$

其中 `output[:,t,:]` 是每个时间步最后一层的 Hidden State。

最终状态：

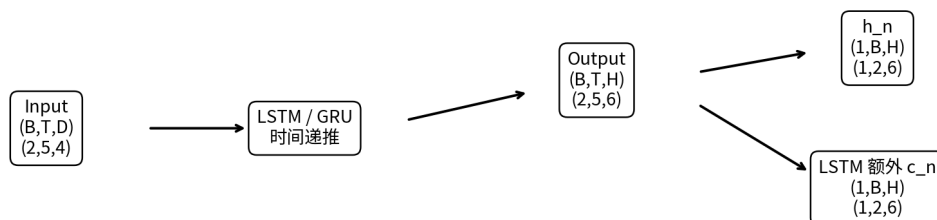
$$h_n : (L \times directions, B, H)$$

$$c_n : (L \times directions, B, H)$$

单层单向时：

$$(1, B, H)$$

LSTM / GRU 的基本 Shape Flow



实验：

$$(B, T, D) = (2, 5, 4), \quad H = 6$$

得到：

$$output = (2, 5, 6)$$

$$h_n = (1, 2, 6), \quad c_n = (1, 2, 6)$$

【常见误区 / 风险】

`output[:, -1, :]` 与 `h_n[-1]` 在最简单的单层单向情容易对应起来，但遇到多层、双向、padding 等设置时不要机械认为所有情况都完全一样。读代码时先确认层数、方向和序列处理方式。

10 GRU：能不能用更少的 Gate 达到类似目的？

GRU 的全称是 **Gated Recurrent Unit**。它没有单独的 Cell State c_t ，而是把状态更新压缩进 Hidden State h_t 。

PyTorch GRUCell 的一种常见写法是：

$$\begin{aligned} r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}) \\ z_t &= \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}) \\ n_t &= \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})) \\ h_t &= (1 - z_t) \odot n_t + z_t \odot h_{t-1} \end{aligned}$$

这里：

- r_t Reset Gate：计算新 Candidate 时，旧 Hidden 要参与多少；
- z_t Update Gate：旧 Hidden 与新 Candidate 怎样混合；
- n_t Candidate Hidden：候选新状态。

【常见误区 / 风险】

不同教材对 GRU 的 z_t 命名和混合公式可能使用互补写法，例如把“更新多少新状态”与“保留多少旧状态”的定义互换。不要只背字母，先看公式： z 大时，到底是更多保留旧状态，还是更多采用新状态？

11 一个 GRU 单步手算

实验使用：

$$x = 0.7, \quad h_{t-1} = 0.2$$

固定 Weight 后得到：

$$r_t \approx 0.581759$$

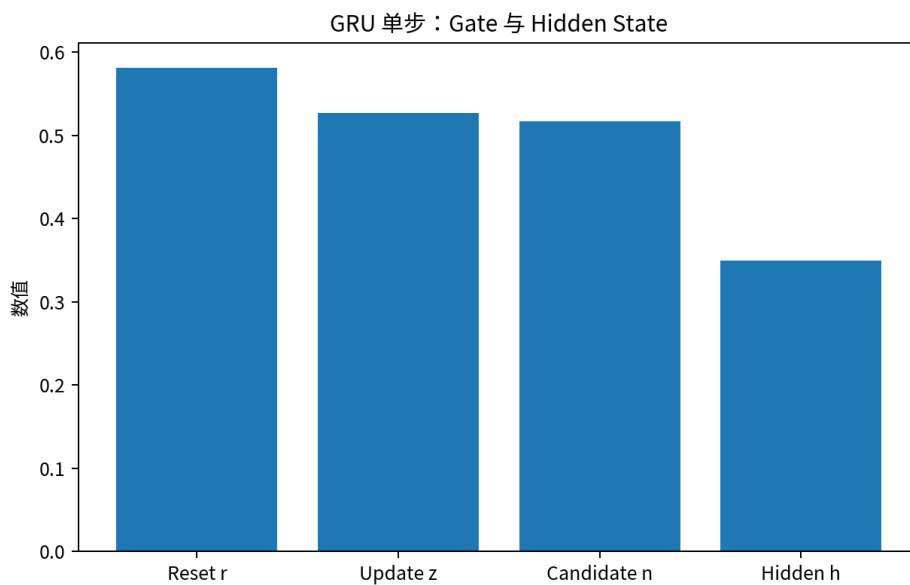
$$z_t \approx 0.527472$$

$$n_t \approx 0.517366$$

$$h_t \approx 0.349964$$

手算与 PyTorch `nn.GRUCell(1,1)` 差异只有约：

$$2.08 \times 10^{-8}$$

**【通俗解释】**

GRU 可以理解为：没有独立 Cell State，而是直接对 Hidden State 做“旧状态保留 + 新 Candidate 写入”的门控混合。结构更紧凑，因此通常比同 Hidden Size 的 LSTM 参数更少。

12 LSTM vs GRU：不要用“谁更高级”理解

比较	LSTM	GRU
核心状态	$h_t + \text{独立 } c_t$	只有 h_t
主要门控	Input / Forget / Output	Reset / Update
Candidate	有 g_t	有 n_t
参数量	约普通 RNN 的 4 倍	约普通 RNN 的 3 倍
状态传递	传 h_t, c_t	传 h_t
结构复杂度	更复杂	更紧凑
是否仍时间递推	是	是

在真实任务中，不能简单预言 LSTM 一定优于 GRU，或者 GRU 一定更快更好。结果还会受到：

- Hidden Size；
- 参数预算；
- Sequence Length；
- 数据规模；
- Learning Rate；
- 初始化；
- Regularization；
- 计算硬件与实现；
- 任务本身。

【科研连接】

如果论文比较 RNN / GRU / LSTM，最少要问：是固定 Hidden Size，还是固定参数量？固定 Hidden Size 时 LSTM 参数更多；固定参数量时 Hidden Size 可能必须不同。两种比较回答的是不同科研问题。

13 状态可以跨 Chunk 传递：这就是流式处理的基础

假设长序列：

$$x_1, x_2, x_3, x_4, x_5$$

可以一次完整处理，也可以拆成：

$$[x_1, x_2]$$

与：

$$[x_3, x_4, x_5]$$

只要第二段把第一段最终状态作为初始状态：

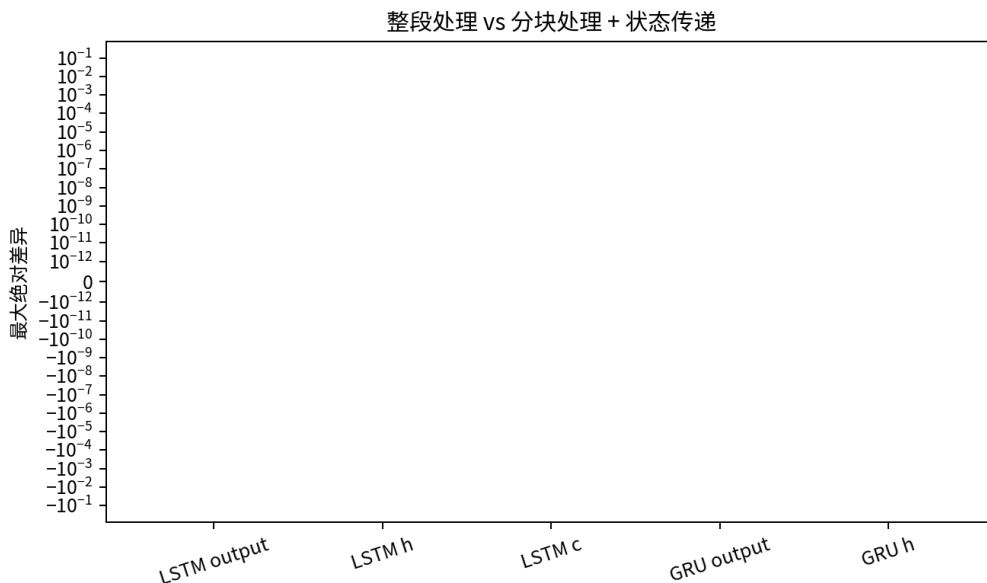
$$(h_2, c_2) \rightarrow \text{第二段 LSTM}$$

理论上就可以延续上下文。

实验中，整段处理与分块处理：

$$\text{LSTM output max diff} = 0$$

GRU output max diff = 0



【通俗解释】

这就是 recurrent state 的工程含义：状态可以作为一个固定维度的“上下文摘要”跨 chunk 传递。它非常适合流式传感器、在线序列、低延迟状态机等场景。

14 但训练长序列时，为什么经常要 detach 状态？——Truncated BPTT

如果你连续处理很多 chunk，同时让每一块状态都保留完整 Autograd 图，那么计算图会从当前 chunk 一直连到很久以前：

$$chunk_1 \rightarrow chunk_2 \rightarrow \dots \rightarrow chunk_K$$

内存与 Backward 成本会不断增长。

因此工程上常使用 **Truncated Backpropagation Through Time**：

```
h = h.detach()
c = c.detach()
```

在某个边界切断更早的 Gradient Graph，但保留数值状态继续 Forward。

【原理】

`detach()` 的意思不是“忘掉 Hidden / Cell 的数值”，而是：**从这个位置开始，不再让新的 Loss 通过 Autograd 追溯到更早的计算图。**

这与 Agent Systems 中“保留状态内容”与“让训练 Gradient 穿过整个历史”是两个不同问题。

15 为什么 LSTM / GRU 仍然难以沿时间完全并行？

无论普通 RNN、LSTM 还是 GRU，都有：

$$state_t = f(x_t, state_{t-1})$$

所以在计算第 t 步前，需要先知道第 $t - 1$ 步状态。

这形成结构依赖链：

$$1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow T$$

GPU 可以并行很多 Batch、Hidden Dimension 中的矩阵运算，但时间步之间本身仍存在递推依赖。

这也是 Transformer 相比 recurrent architecture 的关键改变之一：训练时可以让同一层中的不同 Token 更直接并行计算，再通过 Attention 建立 Token 间关系。

【科研连接】

下一讲会专门比较：RNN/LSTM 的信息路径长度、并行性、长期依赖与 Transformer Attention。到那时你会看到，Transformer 的优势不是简单一句“Attention 比 RNN 强”，而是它改变了序列依赖的计算图结构。

16 LSTM / GRU 与 LLM / Agent Systems 的关系

今天的大型语言模型主干通常不是 LSTM / GRU，但学习它们仍然有价值：

1. 它们是理解“内部状态如何跨时间传播”的最清晰模型；
2. 它们让你理解长期 Gradient Flow 为什么是架构问题，而不只是 Optimizer 问题；
3. 它们帮助理解 Transformer 为什么要减少时间递推依赖；
4. 在流式、小模型、低延迟序列任务中，recurrent state 仍然是一种重要结构思想。

对 Agent Systems，要特别区分：

- RNN/LSTM Hidden State：固定维度、隐式、不可直接解释的内部状态；
- Agent External Memory：可能是文本、数据库、向量库、事件日志，可读写、可检索、可持久化。

【常见误区 / 风险】

不要把“LSTM 有 memory”直接类比成“Agent 有长期记忆系统”。二者都涉及历史信息，但存储介质、容量、读写机制、可解释性和持续时间完全不同。

17 科研实验里怎样公平比较 RNN / GRU / LSTM?

如果做 Ablation，至少记录：

- input size / hidden size;
- 层数与是否 bidirectional;
- 参数量;
- Sequence Length;
- Batch Size;
- Optimizer / LR / Scheduler;
- Gradient Clipping;
- Dropout;
- Training Steps / Tokens Seen;
- 随机种子;
- wall-clock 与显存（如果比较效率）;
- 独立 Test 指标。

还要明确比较预算：

- 固定 Hidden Size?
- 固定 Parameter Count?
- 固定 FLOPs / 时间?
- 固定训练数据量与 Optimizer Steps?

不同控制方式回答不同问题。

18 常见误区

【常见误区 / 风险】

1. “**LSTM 可以永久记住信息。**” 不对。Gate 和有限维状态仍会造成丢失。
2. “**Forget Gate=1 就完全解决梯度消失。**” 不对。真实总梯度还有其他路径，且训练过程复杂得多。
3. “**GRU 是 LSTM 的升级版。**” 不准确。它是另一种更紧凑的 gated recurrent design。
4. “**Gate 是开/关二值开关。**” 不对，通常是连续的软门控。
5. “**LSTM 的 h_t 和 c_t 是同一个东西。**” 不对。 c_t 是独立 Cell State, h_t 是对外 Hidden State。
6. “**有了 LSTM 就没有 BPTT。**” 不对，训练仍需要沿时间做 Backpropagation。
7. “**LSTM 已经像 Transformer 一样可以并行整个序列。**” 不对，时间递推依赖仍存在。

19 本讲最低记忆闭环

你至少要能口述这 8 句话

1. 普通 RNN 只有 Hidden State, LSTM 增加独立 Cell State。
2. Forget Gate 控制旧 Cell State 保留多少。
3. Input Gate 控制新 Candidate 写入多少。
4. Output Gate 控制 Cell State 有多少暴露为 Hidden State。
5. LSTM 的直接 Cell-State 路径使长期信息和 Gradient 更容易保留，但不是无限记忆。
6. GRU 没有独立 Cell State，用 Reset / Update Gate 直接管理 Hidden State。
7. LSTM / GRU 都仍然沿时间递推，所以时间轴并行性有限。
8. 下一步理解 Transformer，要比较的不只是“模型性能”，而是序列依赖的计算图结构。

【科研连接】

下一讲：第 51 讲《为什么 Transformer 最终压过 RNN：长期依赖、路径长度与并行计算》。我们会把 RNN / LSTM / GRU 与 Self-Attention 放在同一张计算结构图里比较，为后面的 Embedding、Q/K/V、Scaled Dot-Product Attention 和完整 Transformer 做最后一次架构过渡。